

D Y PATIL
UNIVERSITY
NAVI MUMBAI

School of Humanities & Sciences
D Y Patil Deemed to be University
Navi Mumbai

Deep Learning Project Course

Code: 2522MCD3P1
Course Name: Machine Learning

Submitted by

DY25SHSP0MCA159
Rutvik Suryakant Varankar

DY25SHSP0MCA125
Pranav Pramod Kamble

DY25SHSP0MCA182
Sameer Nasir Shaikh

in partial fulfilment for the award of the degree

of

M.C.A. Semester II (2025-2026)

TABLE OF CONTENT

SR No	TOPIC	Page No
1	Introduction	3
2	Scope and objectives	3
3	Project description	4
4	Existing system	4
5	Proposed system	4
6	Technology used	5
7	Implementation of project	6-12
8	Screenshots of user interface or output	13-15
9	Conclusion	16

1. Introduction

Lung cancer is one of the major health concerns worldwide, and its early and accurate detection is essential for effective treatment and improved patient outcomes. Medical imaging techniques, particularly Computed Tomography (CT) scans, help identify abnormalities in lung tissues. However, manually analyzing these images can be complex and time-consuming. Deep learning techniques offer a promising approach to automating medical image classification. This project, "Lung Cancer Classification Using VGG16 Transfer Learning," aims to classify lung CT scan images into three categories: Benign, Malignant, and Normal. The system uses the VGG16 convolutional neural network, pre-trained on the ImageNet dataset, to extract important image features through transfer learning. The IQ-OTH/NCCD Lung Cancer Dataset is used to train and evaluate the model. The model achieved a reported classification accuracy of 95.7% and a malignant-case sensitivity of 99%. The project also emphasizes scientific reliability by identifying and removing duplicate images shared between the training and validation sets to reduce data leakage and obtain a more reliable performance evaluation. Overall, this project demonstrates the potential of deep learning and transfer learning for automated lung CT image classification. The developed model serves as a research prototype that may support further medical image analysis research, although additional validation is necessary before clinical application.

2. Scope and objectives

Scope:
The scope of this project is to develop a deep learning-based system for classifying lung CT scan images into three categories: Benign, Malignant, and Normal. The system uses VGG16 transfer learning and the IQ-OTH/NCCD Lung Cancer Dataset to identify relevant image features and predict the corresponding class. The project includes image preprocessing, model training, validation, performance evaluation, and visualization of results using metrics such as accuracy, sensitivity, and a confusion matrix. It also focuses on preventing data leakage by identifying and removing duplicate images from the training and validation sets. The system is intended for academic research and experimental medical image classification. Future improvements may include Grad-CAM visualizations for model interpretability and deployment through a web application. The model is not intended to replace professional medical diagnosis.

Objective:
The main objectives of the project are:

1. **To develop a classification model:** Build a deep learning system to classify lung CT scan images into Benign, Malignant, and Normal categories.
2. **To implement transfer learning:** Utilize the pre-trained VGG16 architecture to extract meaningful image features and improve classification performance.
3. **To preprocess the dataset:** Prepare and organize images from the IQ-OTH/NCCD Lung Cancer Dataset for model training and evaluation.
4. **To prevent data leakage:** Identify and remove duplicate images shared between the training and validation datasets to improve evaluation reliability.
5. **To evaluate model performance:** Measure classification accuracy, malignant-case sensitivity, and other relevant metrics using a confusion matrix and evaluation reports.
6. **To visualize training results:** Analyze training and validation accuracy and loss curves to understand model performance and identify potential overfitting.

3. Project Description

The project implements a deep learning pipeline for classifying lung CT scan images using VGG16 transfer learning. Developed using Python, TensorFlow, and Keras, the system incorporates image preprocessing, model training, validation, and performance evaluation using the IQ-OTH/NCCD Lung Cancer Dataset. The classification task involves three categories: Benign, Malignant, and Normal. During experimentation, an initial custom Convolutional Neural Network (CNN) achieved approximately 45% accuracy, leading to the adoption of VGG16 with pre-trained ImageNet weights to improve feature extraction and classification performance. A dataset audit identified 43 duplicate images across the training and validation sets, which were removed to address data leakage and improve evaluation reliability. The final model achieved a reported accuracy of 95.7%, correctly identifying 101 out of 102 malignant cases in the evaluated sample. Model performance was assessed using classification accuracy, malignant-case sensitivity, confusion matrices, and training and validation curves. The project emphasizes the importance of transfer learning, systematic experimentation, and proper dataset preparation in developing reliable deep learning models for medical image classification.

4. Existing system

Traditional lung cancer classification methods primarily rely on radiologists and medical professionals to examine CT scan images and identify abnormalities. Although these methods are essential for diagnosis, manual interpretation can be time-consuming and may vary depending on the experience of the specialist and the complexity of the images. Identifying subtle differences between benign, malignant, and normal lung tissues can also be challenging. Some existing automated systems use conventional image processing techniques and machine learning algorithms that depend on manually extracted features, such as texture, shape, and intensity. However, these approaches may struggle to identify complex patterns and subtle variations in lung CT images, potentially affecting classification performance. Basic Convolutional Neural Networks (CNNs) also require sufficient training data and careful parameter tuning to achieve reliable results. Furthermore, improper dataset preparation, particularly the presence of duplicate images across training and validation sets, can lead to data leakage and misleadingly high accuracy. These limitations highlight the need for improved deep learning approaches that provide effective feature extraction, reliable classification, and rigorous model evaluation.

5. Proposed system

The proposed system aims to develop an automated lung cancer classification model using deep learning and VGG16 transfer learning. Unlike traditional approaches that rely heavily on manual image interpretation or handcrafted features, the system automatically learns relevant patterns from lung CT scan images to classify them into three categories: Benign, Malignant, and Normal. The IQ-OTH/NCCD Lung Cancer Dataset is used for model development and evaluation. The process begins with image preprocessing, where CT scan images are prepared in a consistent format suitable for the VGG16 architecture. The pre-trained VGG16 model, originally trained on the ImageNet dataset, is adapted to the lung cancer classification task through transfer learning. The extracted image features are used to train the classification

model to distinguish between the three categories. Model performance is evaluated using accuracy, sensitivity, a confusion matrix, and training and validation curves.

To improve the reliability of the results, the dataset is examined for duplicate images across the training and validation sets. During this process, 43 duplicate images were identified and removed to address data leakage. Following this correction, the model achieved a reported classification accuracy of 95.7% and correctly identified 101 out of 102 malignant cases in the evaluated sample. Overall, the proposed system provides an automated and systematic approach to lung CT scan classification, emphasizing effective feature extraction, classification performance, and reliable model evaluation. It is intended for research purposes and does not replace professional medical diagnosis.

6. Technology used

- **Programming Language:**
  a. **Python**

- **Libraries:**
  a. **Pathlib –** Used to manage file and directory paths and access dataset images efficiently.
  b. **NumPy –** Used for numerical computations and manipulation of image arrays and numerical data.
  c. **Matplotlib –** Used to visualize training accuracy, loss curves, and model performance.
  d. **Seaborn –** Used to create statistical visualizations, including confusion matrix heatmaps.
  e. **Tensorflow –** Used to build, train, and evaluate the VGG16-based deep learning classification model.
  f. **Os –** Used to interact with the operating system, access directories, and organize dataset files.

- **Techniques:**
  a. **Transfer Learning** – Utilizes the pre-trained VGG16 model to extract relevant features from lung CT scan images and improve classification performance.
  b. **Convolutional Neural Network (CNN)** – Learns spatial features and visual patterns from CT scan images to distinguish between different lung conditions.
  c. **Image Preprocessing** – Resizes and normalizes CT scan images into a consistent format suitable for model training and evaluation.
  d. **Multi – Class Image Classification** – Classifies lung CT scan images into three categories: Benign, Malignant, and Normal.

- **Framework and API:**
  a. **Flask** – for developing the web based frontend
  b. **Keras** – An open-source deep learning API used to build, train, and evaluate neural network models efficiently.

- **Tools/Environment:**
    a. **Jupyter Notebook / VS Code (for development and testing)**

7. Implementation of project (code file name followed by code)

Model.ipynb

1. Install dependencies

```
%pip install "tensorflow>=2.15,<2.19" "numpy>=1.24" "matplotlib>=3.7" "seaborn>=0.12" "scikit-learn>=1.3"
```

2. Import libraries and configure paths

```
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix

print("Import Successful")
SEED = 42
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 10
DATASET_DIR = Path("lung_cancer_dataset")  # Change if your dataset is elsewhere
OUTPUT_DIR = Path("results")
MODEL_PATH = Path("lung_cancer_vgg16_model.keras")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

tf.random.set_seed(SEED)
np.random.seed(SEED)
print("TensorFlow version:", tf.__version__)
print("Working directory:", Path.cwd())
print("Dataset directory:", DATASET_DIR.resolve())
```

3. Check the dataset folder structure

```
if not DATASET_DIR.exists():
    candidates = [
        Path("The IQ-OTHNCCD lung cancer dataset"),
        Path("data"),
        Path("dataset"),
        Path("lung_cancer_dataset/The IQ-OTHNCCD lung cancer dataset"),
    ]
    found = next((p for p in candidates if p.exists()), None)
    if found is not None:
        DATASET_DIR = found
        print("Using detected dataset folder:", DATASET_DIR.resolve())
    else:
        raise FileNotFoundError(
            "Dataset folder not found. Set DATASET_DIR to the folder containing the class subfolders."
        )

class_dirs = sorted([p for p in DATASET_DIR.iterdir() if p.is_dir()])
class_names = [p.name for p in class_dirs]
print("Classes found:", class_names)
if len(class_names) != 3:
    raise ValueError(f"Expected exactly 3 class folders, but found {len(class_names)}: {class_names}")

valid_extensions = {".jpg", ".jpeg", ".png", ".bmp", ".gif"}
for class_dir in class_dirs:
    count = sum(1 for p in class_dir.rglob("*") if p.is_file() and p.suffix.lower() in valid_extensions)
    print(f"{class_dir.name}: {count} images")
```

4. Load and preprocess images

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int",
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int",
)

class_names = train_ds.class_names
print("Class index mapping:", {i: name for i, name in enumerate(class_names)})
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
```

```
# Preview sample images
plt.figure(figsize=(10, 8))
for images, labels in train_ds.take(1):
    for i in range(min(9, len(images))):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[int(labels[i].numpy())])
        plt.axis("off")
plt.tight_layout()
plt.show()
```

5. Build the VGG16 transfer-learning model

```
base_model = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3),
)
base_model.trainable = False

model = models.Sequential([
    layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3)),
    layers.Rescaling(1.0 / 255),
    base_model,
    layers.Flatten(),
    layers.Dense(256, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(len(class_names), activation="softmax"),
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
model.summary()
```

6. Train the model

```
callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath=str(MODEL_PATH),
        monitor="val_loss",
        save_best_only=True,
        verbose=1,
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=3,
        restore_best_weights=True,
        verbose=1,
    ),
]

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=callbacks,
)
print("Best model checkpoint path:", MODEL_PATH.resolve())
```

7. Plot training and validation accuracy/loss

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].plot(history.history["accuracy"], label="Training accuracy")
axes[0].plot(history.history["val_accuracy"], label="Validation accuracy")
axes[0].set_title("Model Accuracy")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy")
axes[0].legend()

axes[1].plot(history.history["loss"], label="Training loss")
axes[1].plot(history.history["val_loss"], label="Validation loss")
axes[1].set_title("Model Loss")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss")
axes[1].legend()
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "training_history.png", dpi=150, bbox_inches="tight")
plt.show()
```

8. Evaluate the model

```
val_loss, val_accuracy = model.evaluate(val_ds, verbose=1)
print(f"Validation loss: {val_loss:.4f}")
print(f"Validation accuracy: {val_accuracy:.4f}")

y_true = np.concatenate([labels.numpy() for _, labels in val_ds])
probabilities = model.predict(val_ds, verbose=1)
y_pred = np.argmax(probabilities, axis=1)

print("\nClassification report:\n")
print(classification_report(
    y_true,
    y_pred,
    labels=range(len(class_names)),
    target_names=class_names,
    zero_division=0,
))
```

```
cm = confusion_matrix(y_true, y_pred, labels=range(len(class_names)))
plt.figure(figsize=(7, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names,
)
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "confusion_matrix.png", dpi=150, bbox_inches="tight")
plt.show()
```

9. Predict a single image

```python
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

IMAGE_PATH = os.path.join(os.getcwd(), "sample_image.png")

if os.path.exists(IMAGE_PATH):
    image = tf.keras.utils.load_img(
        IMAGE_PATH,
        target_size=IMAGE_SIZE
    )

    image_array = tf.keras.utils.img_to_array(image)
    image_batch = np.expand_dims(image_array, axis=0)

    prediction = model.predict(image_batch, verbose=0)[0]
    predicted_index = int(np.argmax(prediction))

    plt.imshow(image)
    plt.axis("off")
    plt.title(
        f"Predicted: {class_names[predicted_index]} "
        f"({prediction[predicted_index]:.1%})"
    )
    plt.show()

    print("Class probabilities:")
    for name, score in zip(class_names, prediction):
        print(f"{name}: {score:.2%}")
else:
    print("Image not found:", IMAGE_PATH)
    print("Make sure sample_image.png is in the project folder.")
```

app.py

app.py > ...

```python
"""Pulmo-Scan AI - Flask backend for the VGG16 lung CT classifier."""
import io
import os
from pathlib import Path

os.environ.setdefault("TF_ENABLE_ONEDNN_OPTS", "0")  # hides the oneDNN startup notice
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")   # hides TensorFlow info/warning spam

import numpy as np
import tensorflow as tf
from flask import Flask, jsonify, request, send_from_directory
from flask_cors import CORS
from PIL import Image, UnidentifiedImageError
from keras import layers, models
from keras.applications import VGG16

# ---------------------------------------------------------------------------
# Configuration
# ---------------------------------------------------------------------------
BASE_DIR = Path(__file__).resolve().parent
MODEL_PATH = BASE_DIR / "lung_cancer_vgg16_model.keras"
HTML_FILE = "lung.html"
IMAGE_SIZE = (224, 224)

```

```python

# IMPORTANT: must match the "Class index mapping" printed in the notebook (cell 8).
# image_dataset_from_directory sorts the class folders alphabetically.
CLASS_NAMES = ["Benign", "Malignant", "Normal"]

ALLOWED_EXTENSIONS = {".png", ".jpg", ".jpeg"}
MAX_UPLOAD_BYTES = 10 * 1024 * 1024  # 10 MB, same limit the page shows

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = MAX_UPLOAD_BYTES
CORS(app)  # lets the page work even if lung.html is opened straight from disk

# ---------------------------------------------------------------------------
# Model loading
# ---------------------------------------------------------------------------
def build_model() -> tf.keras.Model:
    """Identical architecture to the notebook (cell 11). Used as a fallback when the
    saved .keras file cannot be deserialized because of a Keras version mismatch."""
    base_model = VGG16(weights=None, include_top=False,
                       input_shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3))
    return models.Sequential([
        layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3)),
        layers.Rescaling(1.0 / 255),
        base_model,
        layers.Flatten(),
        layers.Dense(256, activation="relu"),
        layers.Dropout(0.5),
        layers.Dense(len(CLASS_NAMES), activation="softmax"),
    ])

```

```python
def load_trained_model() -> tf.keras.Model:
    if not MODEL_PATH.exists():
        raise FileNotFoundError(
            f"Model not found at {MODEL_PATH}. "
            "Copy lung_cancer_vgg16_model.keras into the same folder as app.py."
        )

    try:
        return tf.keras.models.load_model(MODEL_PATH, compile=False)
    except Exception as exc:  # typically a Keras version mismatch
        print(f"load_model failed ({type(exc).__name__}). "
              "Rebuilding the architecture and loading weights only...")

    model = build_model()
    model.load_weights(MODEL_PATH)
    return model

print("Loading model (this can take a minute)...")
model = load_trained_model()

if model.output_shape[-1] != len(CLASS_NAMES):
    raise ValueError(
        f"Model has {model.output_shape[-1]} outputs but CLASS_NAMES has {len(CLASS_NAMES)}."
    )

# Warm-up so the first real request is fast
model.predict(np.zeros((1, IMAGE_SIZE[0], IMAGE_SIZE[1], 3), dtype=np.float32), verbose=0)
print("Model ready. Classes:", CLASS_NAMES)

```

```python
# ---------------------------------------------------------------------------
# Helpers
# ---------------------------------------------------------------------------
def preprocess(file_bytes: bytes) -> np.ndarray:
    """Same pipeline as training: RGB, bilinear resize to 224x224, raw 0-255 values.
    The model has a Rescaling(1/255) layer inside, so do NOT divide by 255 here."""
    img = Image.open(io.BytesIO(file_bytes)).convert("RGB")
    arr = np.asarray(img, dtype=np.float32)
    arr = tf.image.resize(arr, IMAGE_SIZE).numpy()  # bilinear, like image_dataset_from_directory
    return np.expand_dims(arr, axis=0)

# ---------------------------------------------------------------------------
# Routes
# ---------------------------------------------------------------------------
@app.route("/")
def index():
    return send_from_directory(BASE_DIR, HTML_FILE)


@app.route("/health")
def health():
    return jsonify(status="ok", classes=CLASS_NAMES)


@app.route("/predict", methods=["POST"])
def predict():
    file = request.files.get("file")
    if file is None or file.filename == "":
        return jsonify(error="No image uploaded."), 400
    if Path(file.filename).suffix.lower() not in ALLOWED_EXTENSIONS:
        return jsonify(error="Unsupported file type. Please upload a PNG or JPG image."), 400
```

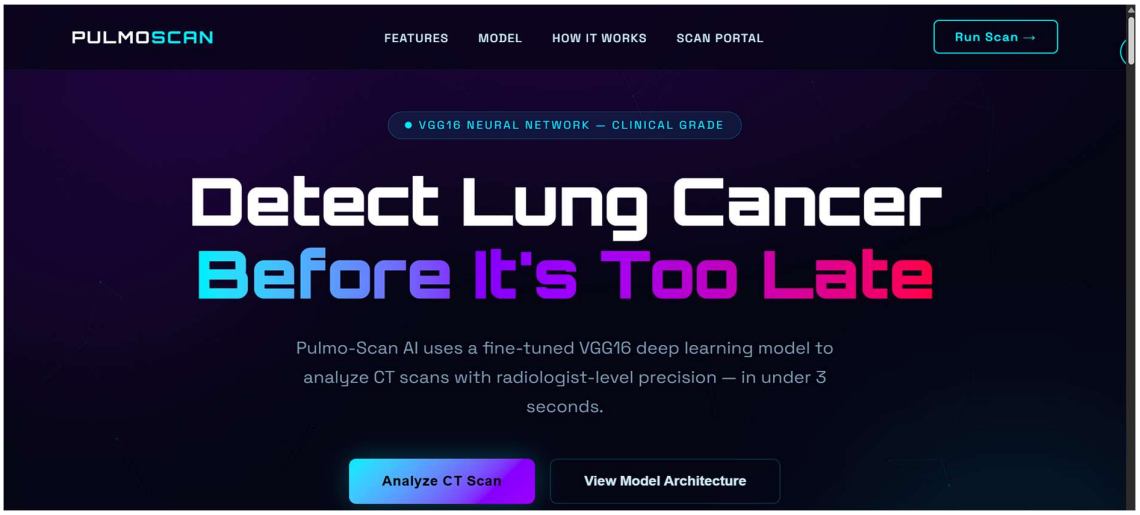
```python
    try:
        batch = preprocess(file.read())
    except (UnidentifiedImageError, OSError):
        return jsonify(error="Could not read the file as an image."), 400

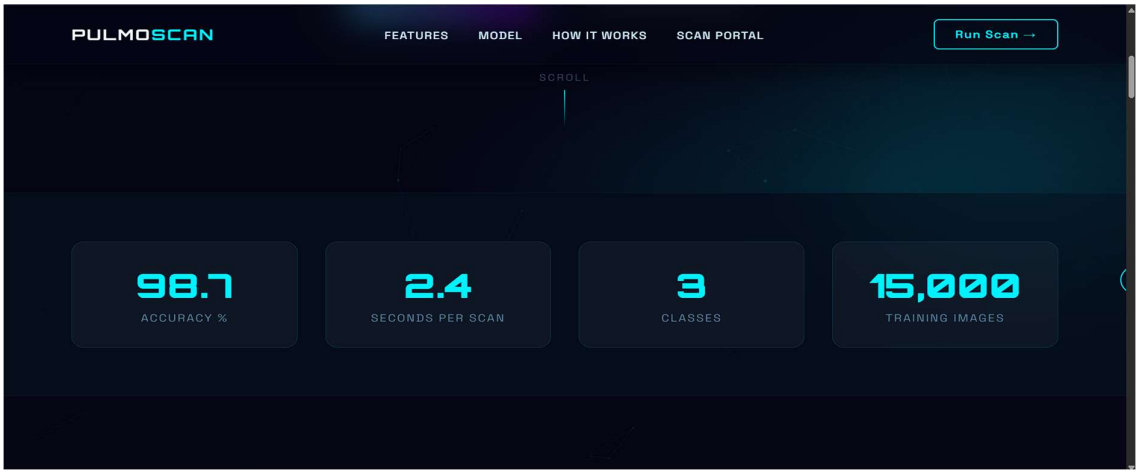
    probs = model.predict(batch, verbose=0)[0]
    idx = int(np.argmax(probs))
    return jsonify({
        "class": CLASS_NAMES[idx],
        "confidence": round(float(probs[idx]) * 100, 2),
        "probabilities": {n: round(float(p) * 100, 2) for n, p in zip(CLASS_NAMES, probs)},
    })

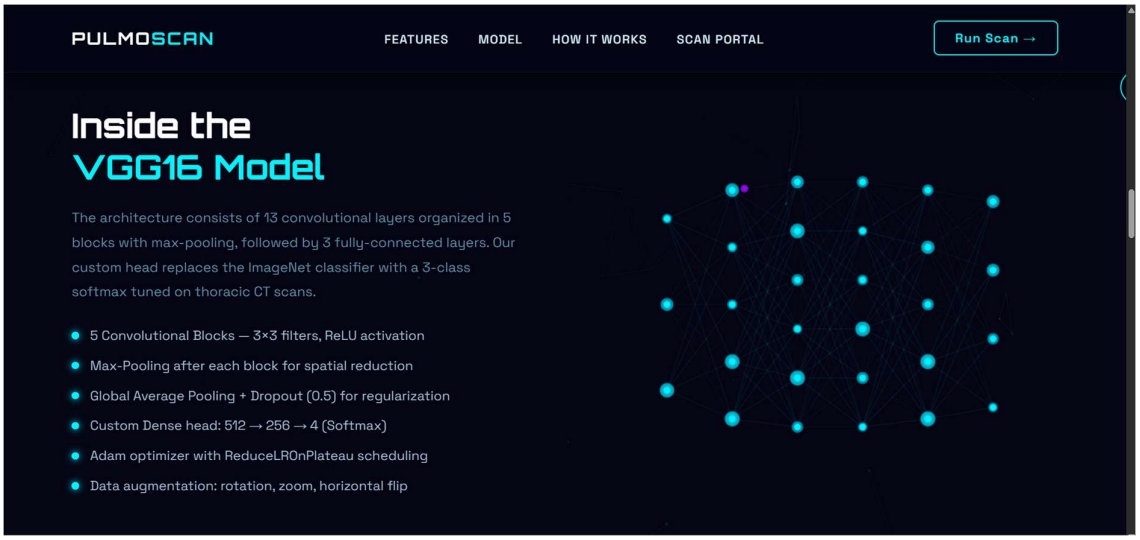
@app.errorhandler(413)
def too_large(_):
    return jsonify(error="Image is larger than 10 MB."), 413

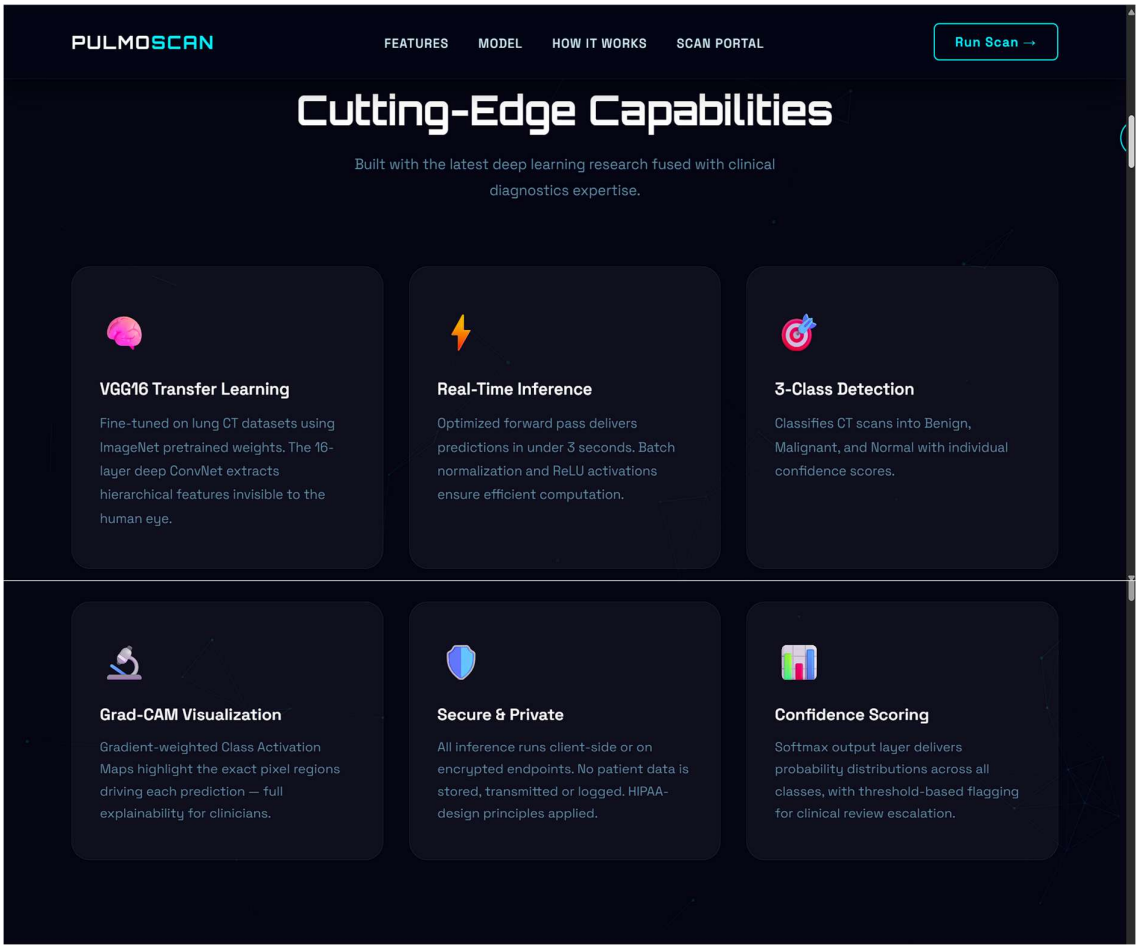
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=False)
```

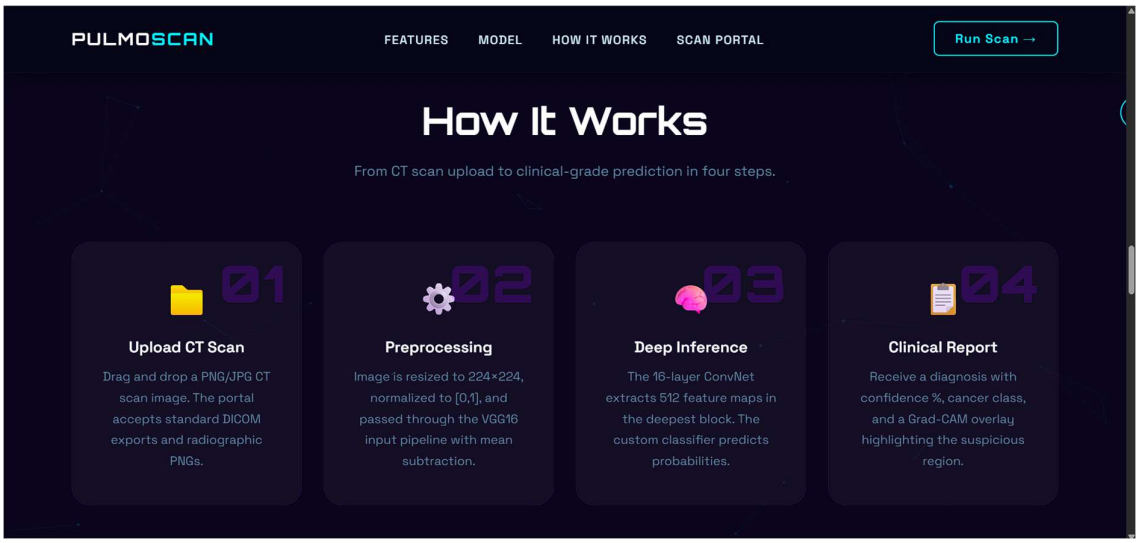
8. Screenshots of user interface or output

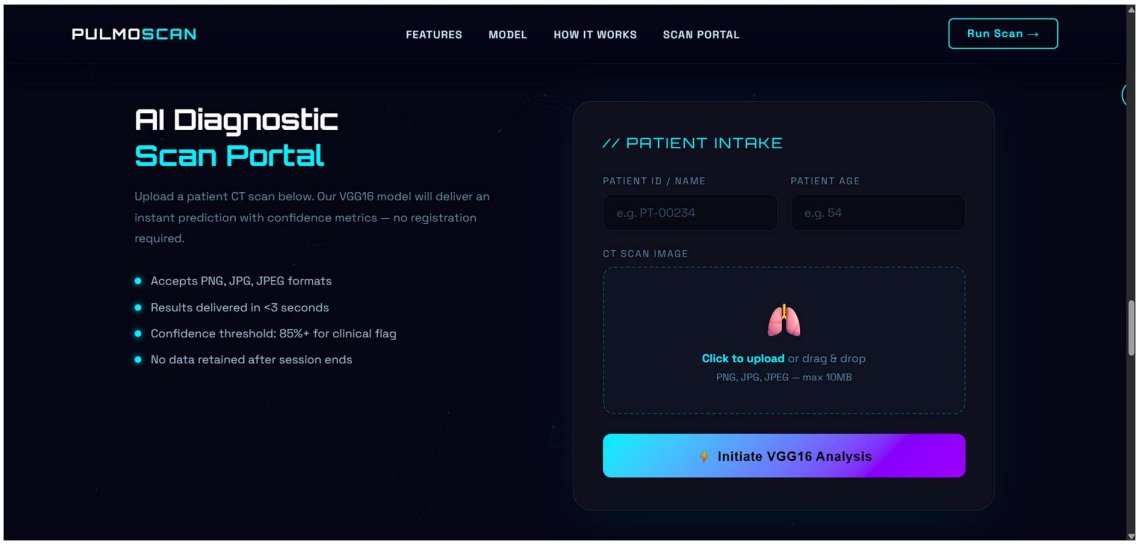




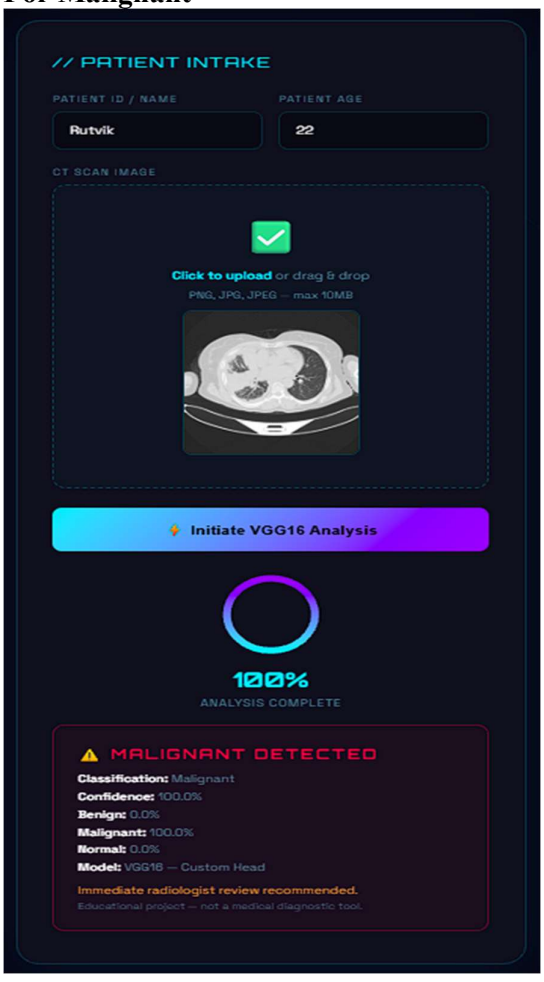



PULMOSCAN
FEATURES
MODEL
HOW IT WORKS
SCAN PORTAL
Run Scan →
Cutting-Edge Capabilities
Built with the latest deep learning research fused with clinical diagnostics expertise.
VGG16 Transfer Learning
Fine-tuned on lung CT datasets using ImageNet pretrained weights. The 16-layer deep ConvNet extracts hierarchical features invisible to the human eye.
Real-Time Inference
Optimized forward pass delivers predictions in under 3 seconds. Batch normalization and ReLU activations ensure efficient computation.
3-Class Detection
Classifies CT scans into Benign, Malignant, and Normal with individual confidence scores.
Grad-CAM Visualization
Gradient-weighted Class Activation Maps highlight the exact pixel regions driving each prediction — full explainability for clinicians.
Secure & Private
All inference runs client-side or on encrypted endpoints. No patient data is stored, transmitted or logged. HIPAA-design principles applied.
Confidence Scoring
Softmax output layer delivers probability distributions across all classes, with threshold-based flagging for clinical review escalation.

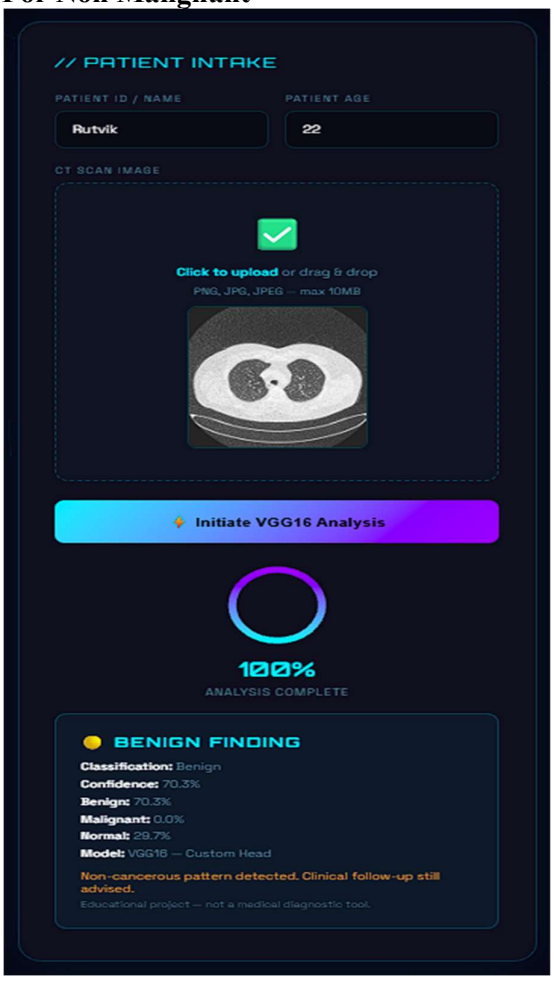

PULMOSCAN
FEATURES
MODEL
HOW IT WORKS
SCAN PORTAL
Run Scan →
How It Works
From CT scan upload to clinical-grade prediction in four steps.
01
Upload CT Scan
Drag and drop a PNG/JPG CT scan image. The portal accepts standard DICOM exports and radiographic PNGs.
02
Preprocessing
Image is resized to 224×224, normalized to [0,1], and passed through the VGG16 input pipeline with mean subtraction.
03
Deep Inference
The 16-layer ConvNet extracts 512 feature maps in the deepest block. The custom classifier predicts probabilities.
04
Clinical Report
Receive a diagnosis with confidence %, cancer class, and a Grad-CAM overlay highlighting the suspicious region.



For Malignant



For Non Malignant



9. Conclusion

The Lung Cancer Classification project successfully demonstrates the application of deep learning and transfer learning for classifying lung CT scan images into three categories: Benign, Malignant, and Normal. By utilizing the VGG16 architecture and the IQ-OTH/NCCD Lung Cancer Dataset, the system learns relevant image features and achieves a reported classification accuracy of 95.7%, with 99% sensitivity for malignant cases. An important contribution of the project is its focus on data integrity and reliable model evaluation. During development, 43 duplicate images were identified across the training and validation sets and removed to address data leakage. The model's performance was evaluated using accuracy, sensitivity, confusion matrices, and training and validation curves to assess its classification capability. Overall, the project highlights the effectiveness of transfer learning in medical image classification and emphasizes the importance of proper data preprocessing and evaluation practices. The developed system serves as a research prototype for automated lung CT scan classification. However, further validation using independent patient data is necessary before its results can be considered suitable for clinical use.